TEMCO LEARN

T3000 API & Protocol

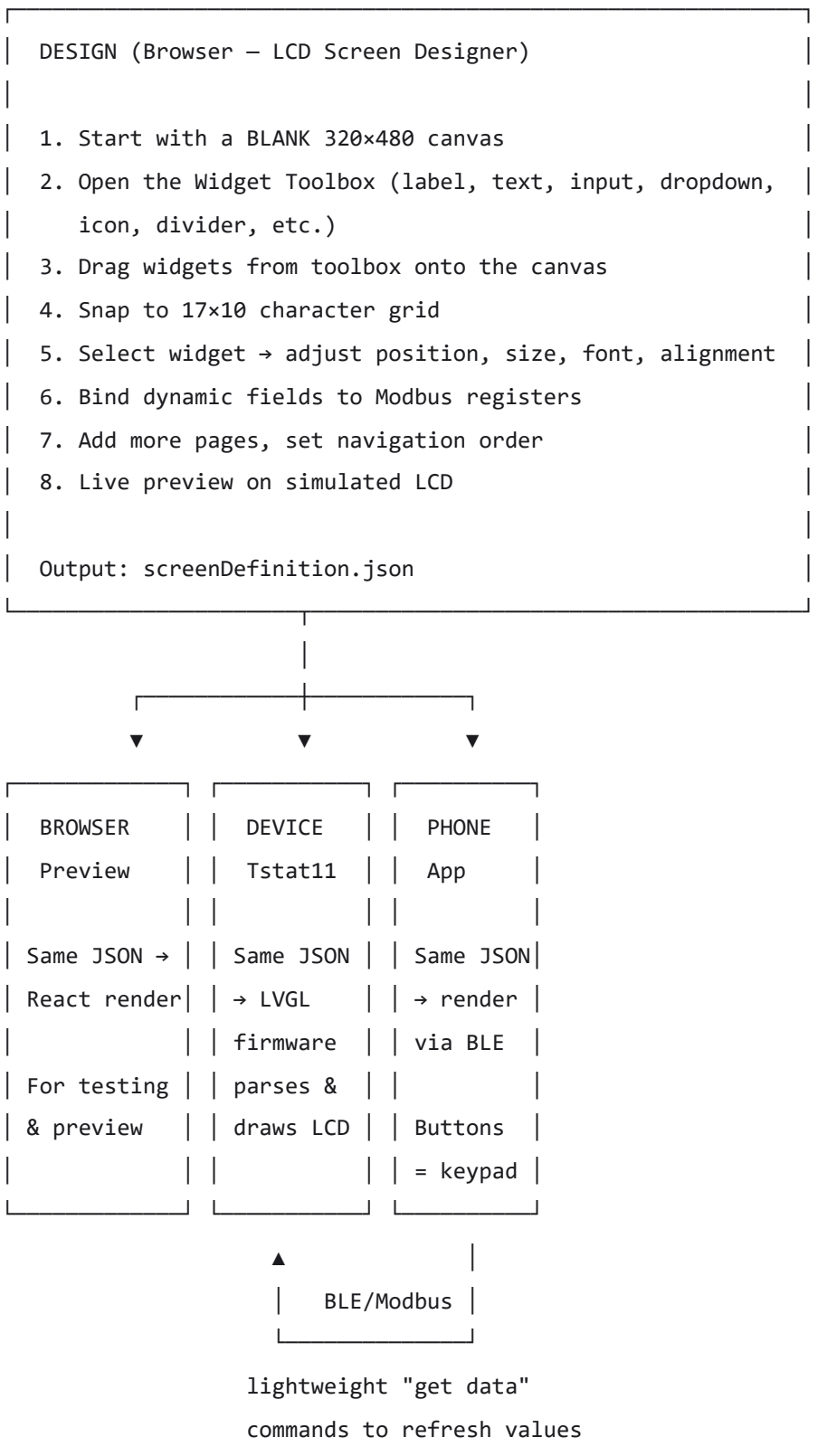
System Overview · built Nov 13, 2026

System Overview

End-to-End Architecture

The Tstat LCD system uses a **single JSON definition** to drive LCD rendering across three platforms: browser simulator, physical device, and phone app.

TEXT


DESIGN (Browser — LCD Screen Designer)
1. Start with a BLANK 320×480 canvas
2. Open the Widget Toolbox (label, text, input, dropdown, icon, divider, etc.)
3. Drag widgets from toolbox onto the canvas
4. Snap to 17×10 character grid
5. Select widget → adjust position, size, font, alignment
6. Bind dynamic fields to Modbus registers
7. Add more pages, set navigation order
8. Live preview on simulated LCD
Output: screenDefinition.json
BROWSER
Preview
Same JSON → React render
For testing & preview
DEVICE
Tstat11
Same JSON → LVGL firmware parses & draws LCD
PHONE
App
Same JSON → render via BLE
Buttons = keypad
BLE/Modbus
lightweight "get data" commands to refresh values

Core Principles

1. Blank Canvas + Toolbox Design

The LCD designer starts with an **empty 320×480 canvas** (not pre-built templates). The designer drags widgets from a toolbox onto the canvas, snaps them to the character grid, and adjusts their properties. This is similar to how form designers or LVGL GUI builders work.

From the original design spec (lcd-menu.md):

> "The only HTML should be the empty #tstat-lcd container. Everything else must be generated by JavaScript from the JSON description."

The existing hardware screens (Main, Setpoint, Inside, Comm Settings) are just **example layouts** that were designed this way. Any custom layout can be created from scratch.

2. No Firmware Flash for UI Changes

> "User-defined menu systems will be converted into JSON strings (or a compressed byte-version). These strings will be chunked and sent to the Tstat10 via Modbus Register Writes. The Tstat10 firmware will receive the text, parse the LVGL instructions, and rebuild the local screen menu without a firmware flash."

3. One Definition, Three Renderers

The same `screenDefinition.json` is consumed by:

- **Browser simulator** — React components render the JSON
- **Device firmware** — ESP32 parses JSON, draws via LVGL on 320×480 LCD
- **Phone app** — Renders from local resources, sends lightweight commands via BLE

4. Phone App Knows All Temco Devices

The phone app ships with built-in knowledge of all Temco device UIs. It only needs to send lightweight "get data" commands to refresh values. The full UI structure is rendered locally from the JSON definition.

Integration Points

A. T3000 Desktop (Windows)

- T3000 hosts the simulator via **WebView2 (Chromium)**
- Data flow: T3000 polls device over BACnet/Modbus → pushes updates via `ExecuteScriptAsync()` → calls `window.updateUI(json)`

B. T3000 Web / T3000Webview

- Simulator runs as a native web component or iframe
- Compatible with the broader T3000Webview dashboard

C. Device Firmware (ESP32)

- Fixed coordinate rendering at 320×480
- No LVGL library used — firmware draws directly to frame buffer
- JSON parsed at runtime to position text/values

D. Phone App (BLE)

- Auto-connects to strongest RSSI Temco device
- If out of range, shows nearby top 5 devices
- Returns to original device when back in range
- Renders LCD UI locally from JSON definition
- Button presses sent as keypad commands via BLE

Hardware Specs

Spec	Value
LCD Resolution	320 × 480 px
Character Grid	17 columns × 10 rows
Font	Bold monospace
Background	#003366 (Temco Blue)
Highlight	#008080 (Teal)
Buttons	4 physical: ◄ ▼ ▲ ►
BLE Support	32M devices only (high ROM/RAM usage)

Data Flow Summary

TEXT

```
Design Time:
  Blank Canvas → Drag widgets from Toolbox → Snap to grid →
  Adjust properties → Preview → Export screenDefinition.json

Runtime (Device):
  screenDefinition.json → Modbus register writes → ESP32 firmware → LCD

Runtime (Phone):
  BLE auto-connect → "get data" command → device returns values
  Phone renders UI from local JSON + fresh values

Runtime (Browser/T3000):
  T3000 polls device → updateUI(json) → WebView2 renders
```